Kaarigar Setu

Implementation screenshots

Screens of the working system and the output of its security tooling, each with a note on what it shows and which implemented feature it demonstrates. Captured on 19 October 2026 from the development stack running on this computer.

How to read the evidence.

Figures 1 to 14 (web application) are real screenshots of the running website in a browser, taken just now. The people shown (Ramesh Kumar and the others) are fictional demo accounts created by the demo seed; they are not real workers.

Figures 15 to 18 (moderator console) are rendered by the console's own screenshot mode and filled with clearly marked DEMO rows (the console cannot be screenshotted headlessly any other way). Their data is fictional.

Figures 19 to 28 (command-line evidence) are the real text output of the named commands, drawn into an image so that Hindi text displays correctly. Figure 19 comes from the last complete attack run (16 October) and Figure 28 from the saved log of the last full end-to-end run; all others were run today.

Every image is also saved as a file in `docs/screenshots/` in the project.

Not shown, because it was not run or could not be captured here: a real phone scanning the QR code or recording audio over HTTPS; the production Docker and HTTPS proxy; the CI pages on GitHub (OWASP ZAP, Trivy); the language-model scorer (no API key); real SMS delivery; and the message the page shows when the microphone is used over an insecure address (the address was not reachable from the capture browser at the time).

Contents

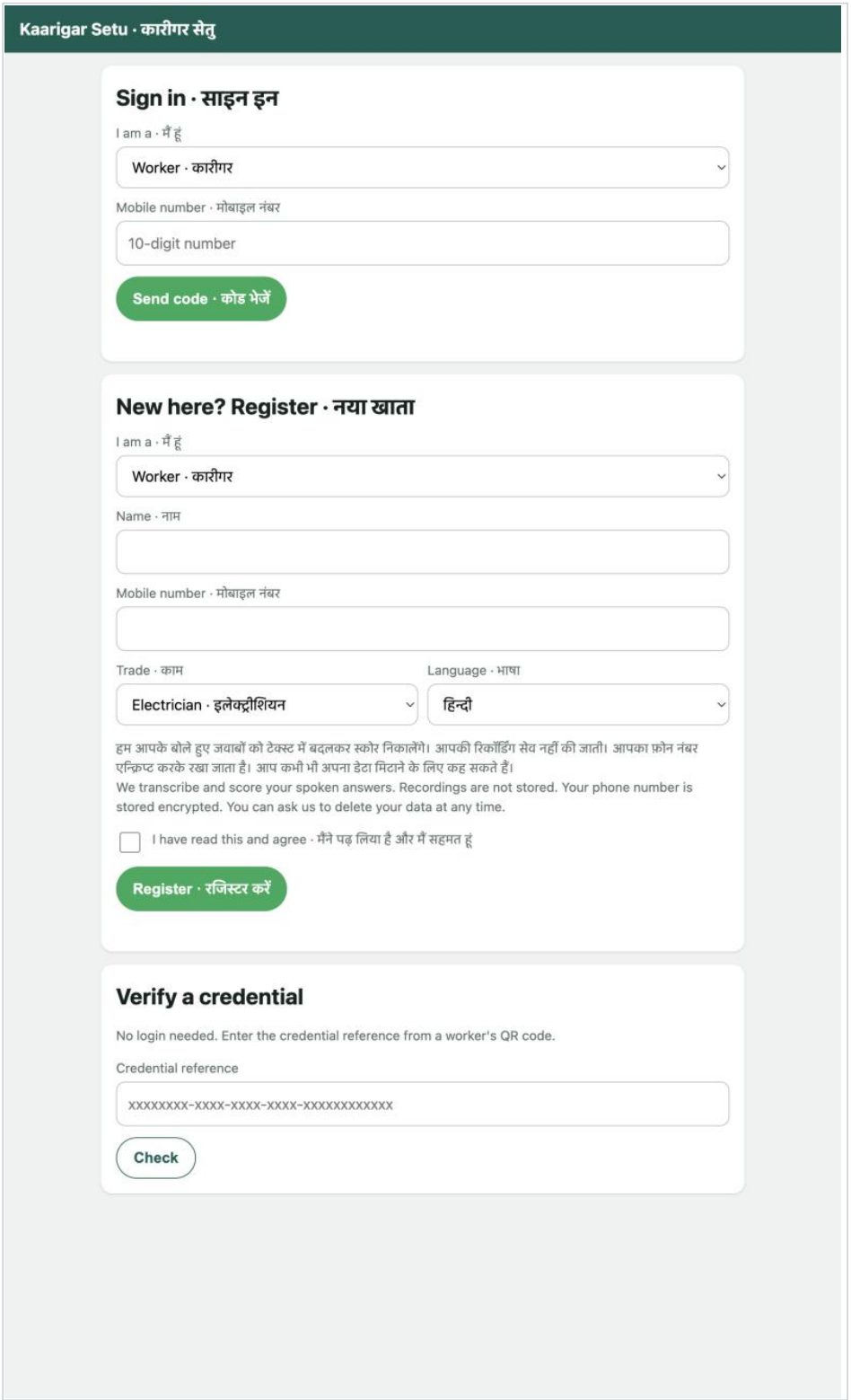
Fig.	Screen or output	Implemented feature it demonstrates
1	Sign in and registration	Consent box must be ticked or nothing is stored; sign-up is throttled to 20 per 10 minutes per address; code requests are limited per number and per a...
2	Demo accounts (development only)	This page exists only when DEMO_LOGIN=true and APP_ENV=dev, and it refuses real accounts. The employers shown as verified were vetted by the demo seed...
3	Worker's Skill and Trust Passport	Trust score built from employer confirmations (distinct employers, recency, credibility), explained to the worker. The Excellent band requires at leas...
4	Latest assessment and the signed credential	The credential is an RSA-signed document, checked again on every verification (signature, key id, revocation, expiry). It carries a random pseudonym, ...
5	Live presentation code in use	The code is an HMAC token tied to one credential, minted only by the signed-in owner; it lets a verifier see the credential is being shown by its owne...
6	Employer confirmations and the right to delete	Employer confirmations feed the trust score. Deletion removes the profile, transcripts, scores and credentials and ends the session (DPDP deletion pat...
7	Public verification page: fixed link	Verification is public, throttled to 120 requests a minute per address, and re-checks everything on the server. The warning box is the honest answer t...
8	Public verification page: live code	The link was minted through the API, decoded from the live QR and opened here; the page recomputes the HMAC and the age.
9	Public verification page: made-up code	A token that is not a valid HMAC for this credential is rejected; an old token shows 'Not live', a missing one shows 'Static link'.
10	Verification page at phone width	Responsive layout with a strict content-security policy (no inline scripts) so the page works on a phone browser.
11	Voice assessment: introduction	While preparing these screenshots a mismatch was found and fixed: the Hindi sentence said 5 questions while the English said 10. This capture is after...
12	Voice assessment: question read aloud	Only the question text and marks are sent to the browser, never the grading keywords. The rubrics are placeholders until a trade expert supplies real ...
13	Voice assessment: ready to answer	The recording is converted in the browser to 16 kHz WAV, then size, header and duration are validated on the server before anything else; the audio is...
14	Employer dashboard	A confirmation needs a one-time code sent to the employer's own phone, then passes the fraud model; self-attestation is refused and volume is rate-lim...
15	Console sign-in	Password hashed with PBKDF2 (210,000 rounds); unknown usernames cost the same time as wrong passwords; five attempts per ten minutes.
16	Fraud review queue	Human in the loop for the fraud model. A flag can be decided only once and every decision is written to the audit trail.
17	Employers tab: vetting	Vetting is the trust anchor of the Excellent band; it records who and when and refreshes trust scores. This closed the weakness where three fake emplo...
18	Security tab	CSRF rejections, role violations and rate-limit hits are audited (once per client per minute) and compared with thresholds; the integrity line checks ...

Fig.	Screen or output	Implemented feature it demonstrates
19	Attack-and-defence run: 40 attacks	scripts/security_attack_demo.py attacks the running stack: injection, sessions, access control, CSRF, credential forgery, data exposure, scoring abuse...
20	Offline credential verification: passes	scripts/verify_credential.py shares no code with the application. Signing uses RFC 8785 canonical form so any JSON tool can rebuild the signed bytes.
21	Offline credential verification: edited credential rejected	Any edit to a signed field breaks verification, whether it is made to the displayed credential or to the signed text.
22	Audit trail attacks	Hash chain + keyed head record + anchors of the newest line pinned in the append-only database every 25 events and every 5 minutes.
23	Fake-employer simulation	Run against the production scoring code. Three throwaway accounts used to be enough for Excellent; now the top band is unreachable without a moderator...
24	Public key list and signed revocation list	Retired keys stay listed after rotation so old credentials keep verifying; the revocation list is signed so a proxy cannot add, remove or re-date entr...
25	Speech recognition (8-bit Whisper) and the service token	The internal scoring service only answers callers that present the shared token. Measured accuracy on real voices is lower than on this clean syntheti...
26	Test suites	Tests include the guard that fails the build if any database query is built by joining strings, the SQL-injection fuzz over every data-access method, ...
27	Static analysis, secret and dependency scans	Findings from earlier runs were fixed or justified in writing (docs/SECURITY_SCAN_REPORT.md). The same checks are defined as CI jobs, which have not y...
28	End-to-end suite: the security sections	Runs against the live application with the labelled test transcriber, so the voice step uses synthetic audio rather than real speech.

Part A. The web application

Pages as a worker, an employer and a stranger checking a credential would see them.

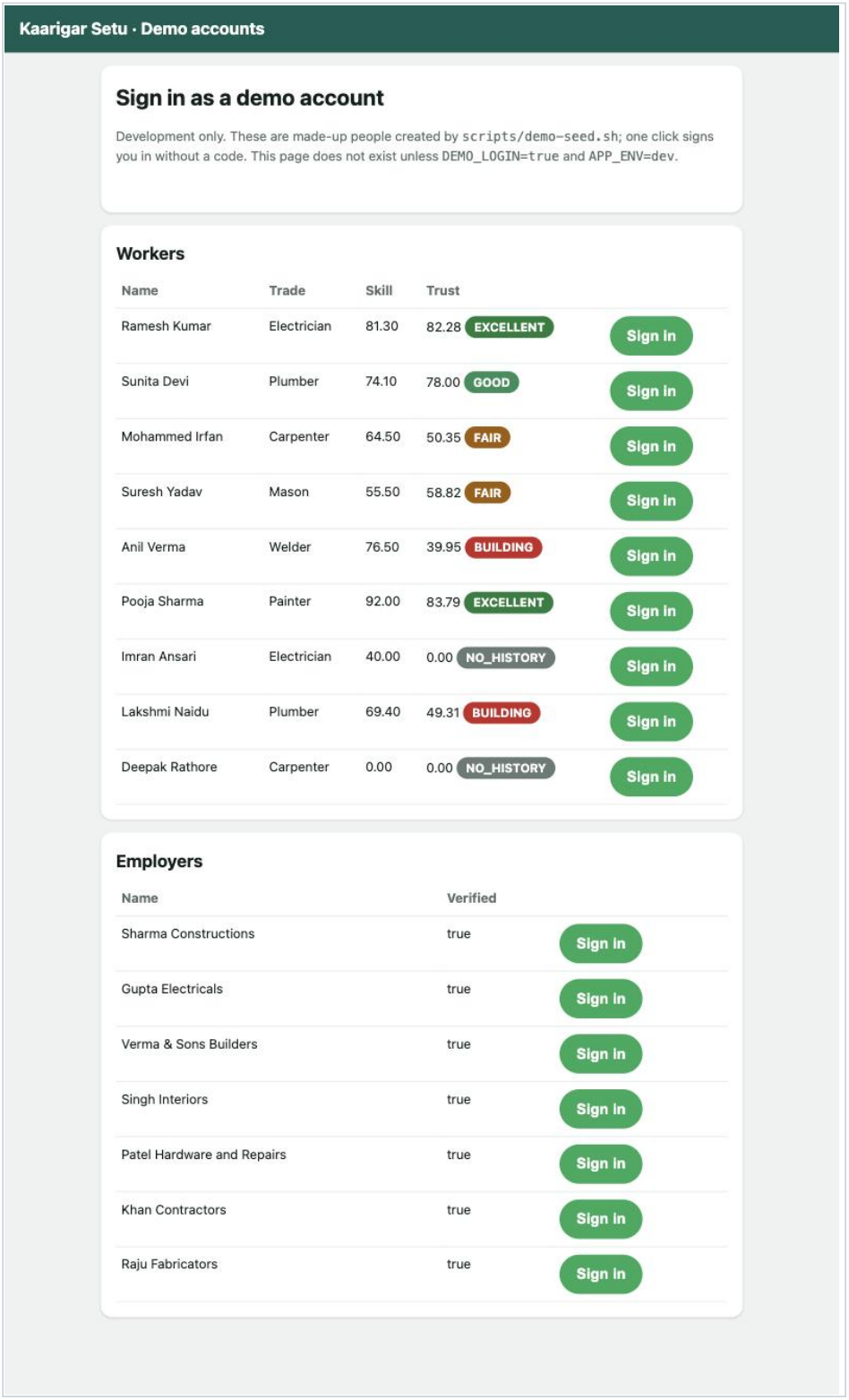
Figure 1. Sign in and registration



What it shows. Bilingual (Hindi and English) sign-in by mobile number and one-time code, a registration form for workers and employers with the privacy notice, and a box to check a credential reference without signing in.

Implementation point. Consent box must be ticked or nothing is stored; sign-up is throttled to 20 per 10 minutes per address; code requests are limited per number and per address; the phone number is stored encrypted.

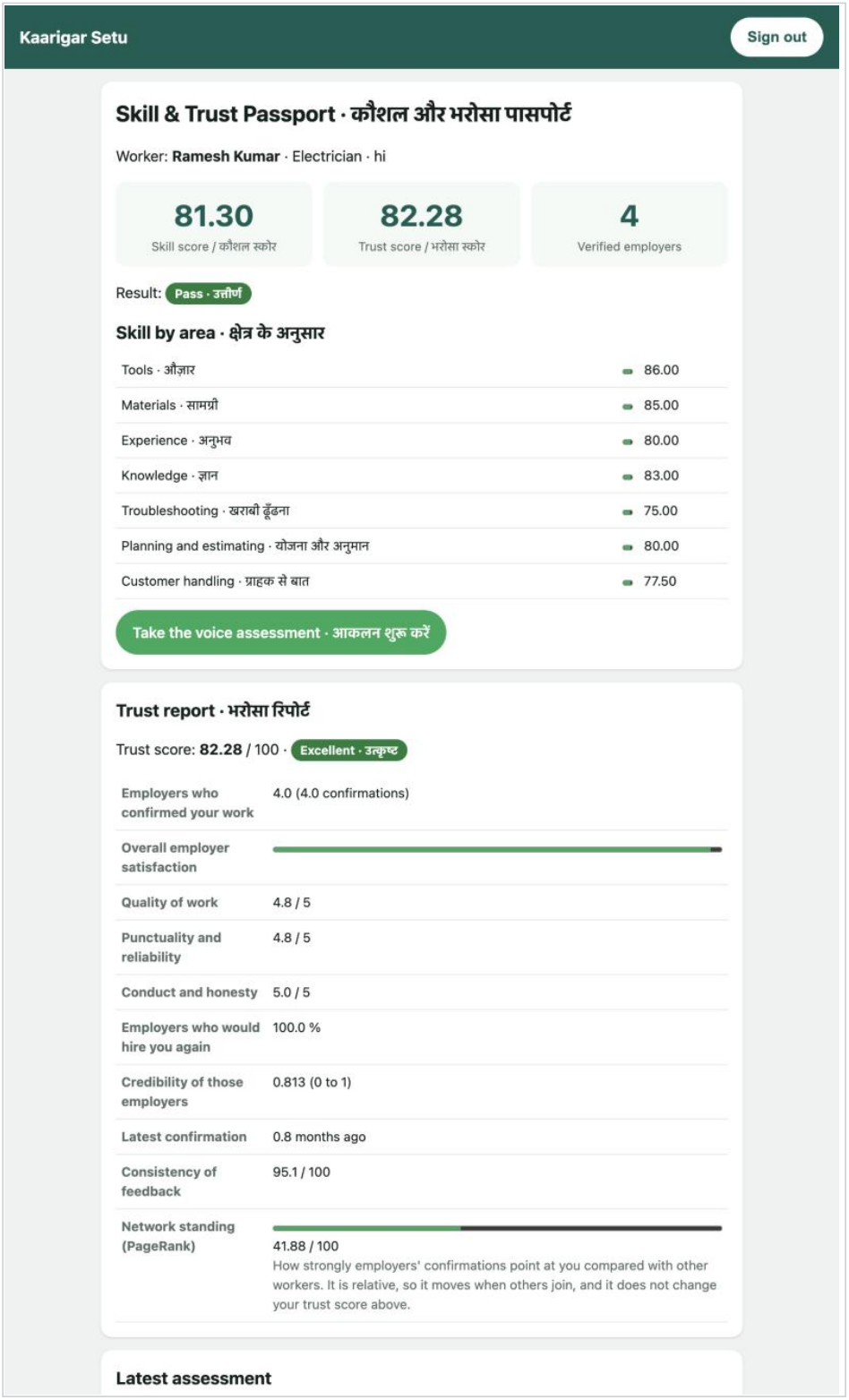
Figure 2. Demo accounts (development only)



What it shows. One-click sign-in as fictional workers and employers, with their skill scores and trust bands (Excellent, Good, Fair, Building, No history).

Implementation point. This page exists only when DEMO_LOGIN=true and APP_ENV=dev, and it refuses real accounts. The employers shown as verified were vetted by the demo seed; in real use only a moderator can vet an employer.

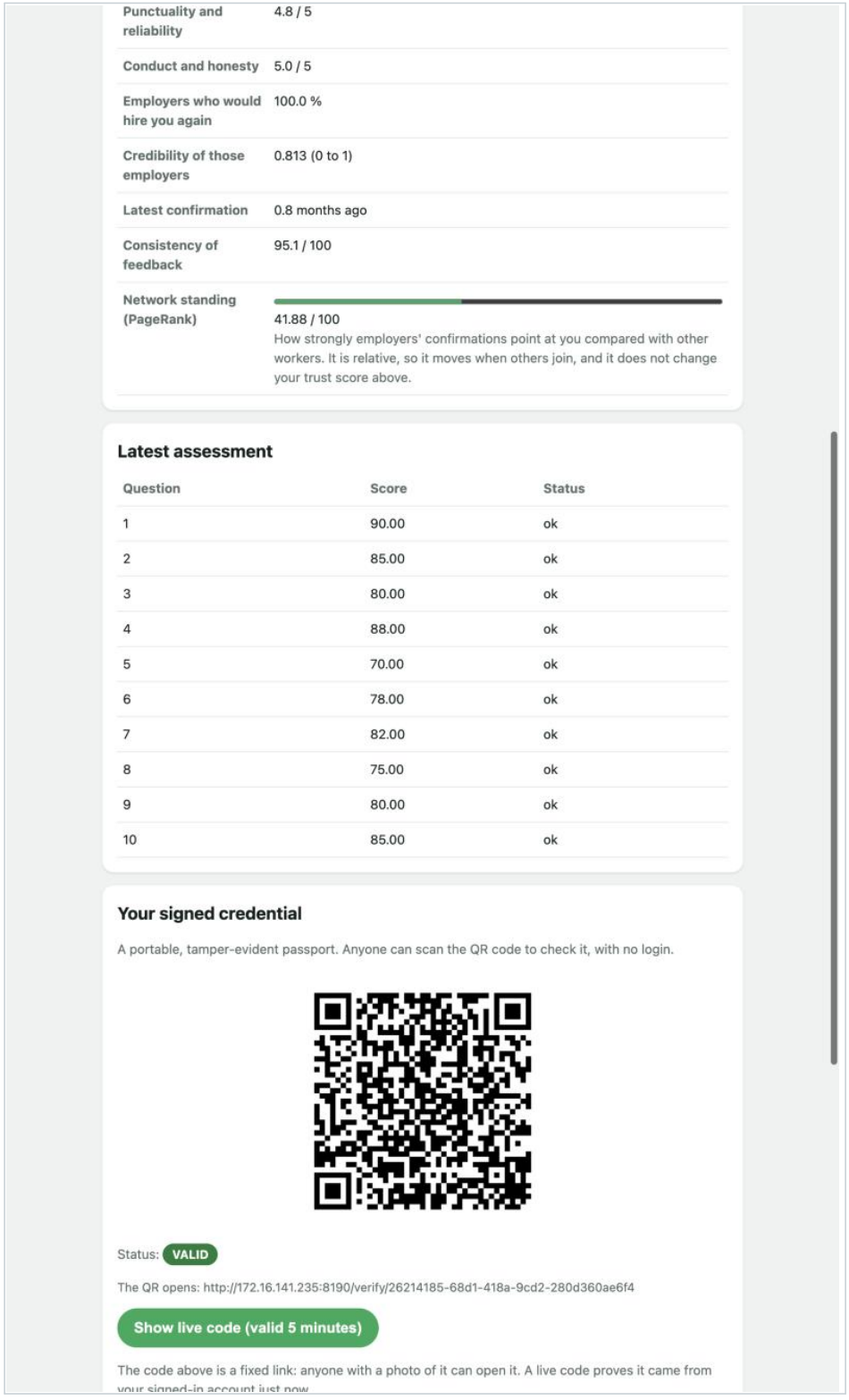
Figure 3. Worker's Skill and Trust Passport



What it shows. The skill score with a result band and a breakdown by area (tools, materials, experience, knowledge, troubleshooting, planning, customer handling), and the trust report: satisfaction, ratings, would-hire-again rate, employer credibility, consistency, and network standing from the endorsement graph (PageRank).

Implementation point. Trust score built from employer confirmations (distinct employers, recency, credibility), explained to the worker. The Excellent band requires at least one employer a moderator has vetted.

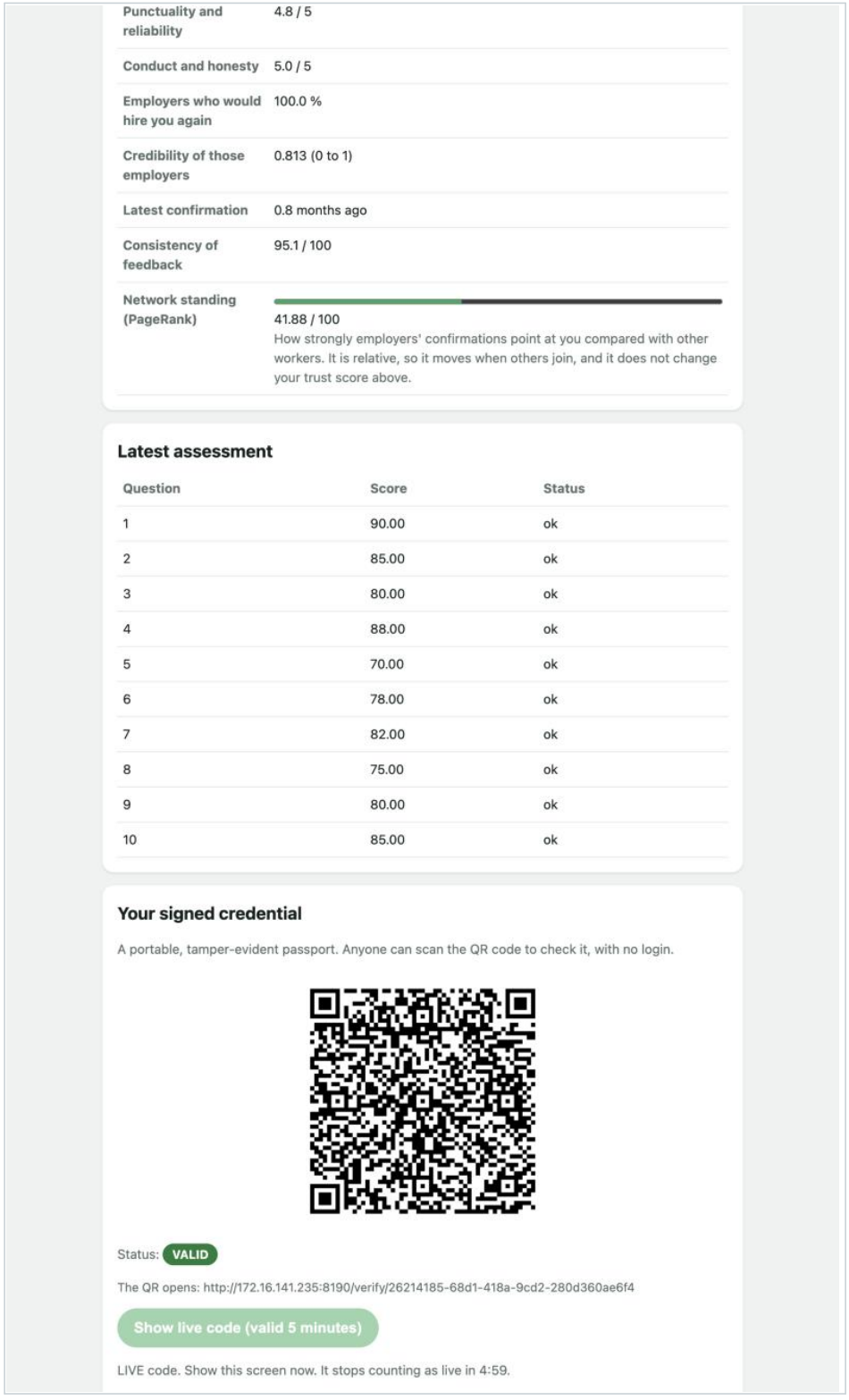
Figure 4. Latest assessment and the signed credential



What it shows. The ten per-question scores, then the credential: a QR code, its status (VALID), the address the QR opens, and a button for a live code.

Implementation point. The credential is an RSA-signed document, checked again on every verification (signature, key id, revocation, expiry). It carries a random pseudonym, never a name or phone number.

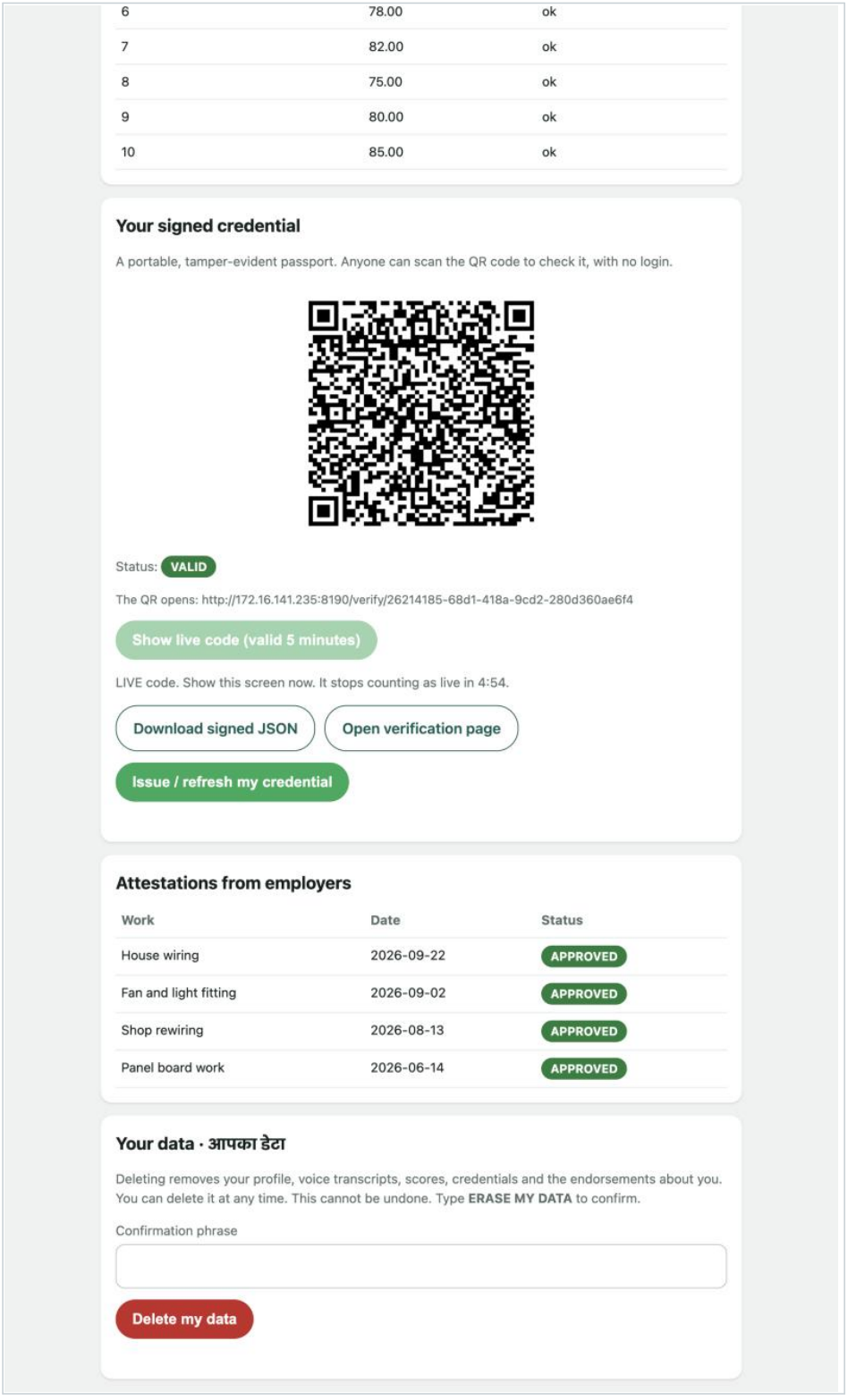
Figure 5. Live presentation code in use



What it shows. After pressing the button, the QR changes to one that carries a 5-minute live code, the button is disabled, and a countdown reads 4:59.

Implementation point. The code is an HMAC token tied to one credential, minted only by the signed-in owner; it lets a verifier see the credential is being shown by its owner right now.

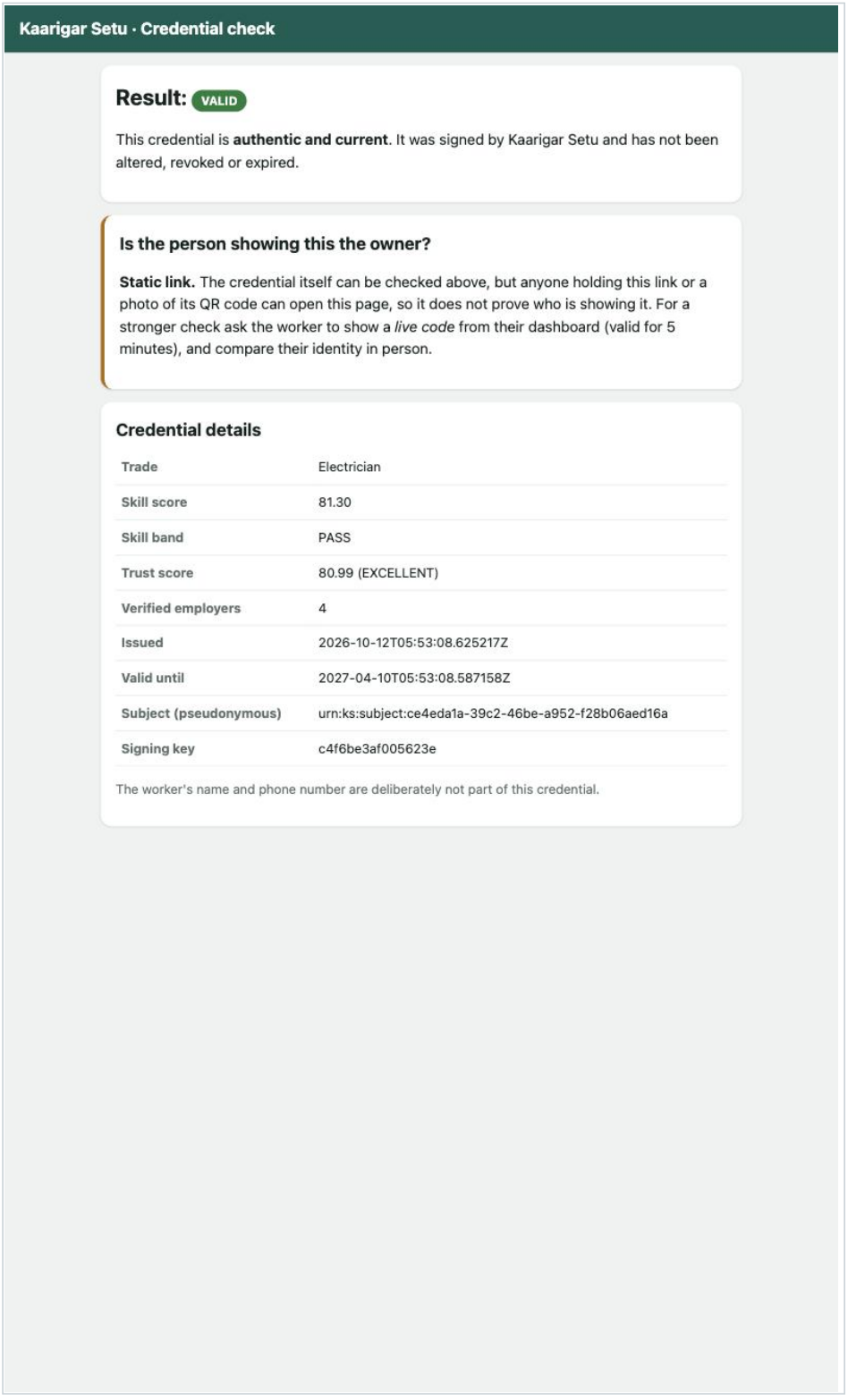
Figure 6. Employer confirmations and the right to delete



What it shows. The list of approved employer confirmations, and the data deletion box that requires typing the exact confirmation phrase.

Implementation point. Employer confirmations feed the trust score. Deletion removes the profile, transcripts, scores and credentials and ends the session (DPDP deletion path).

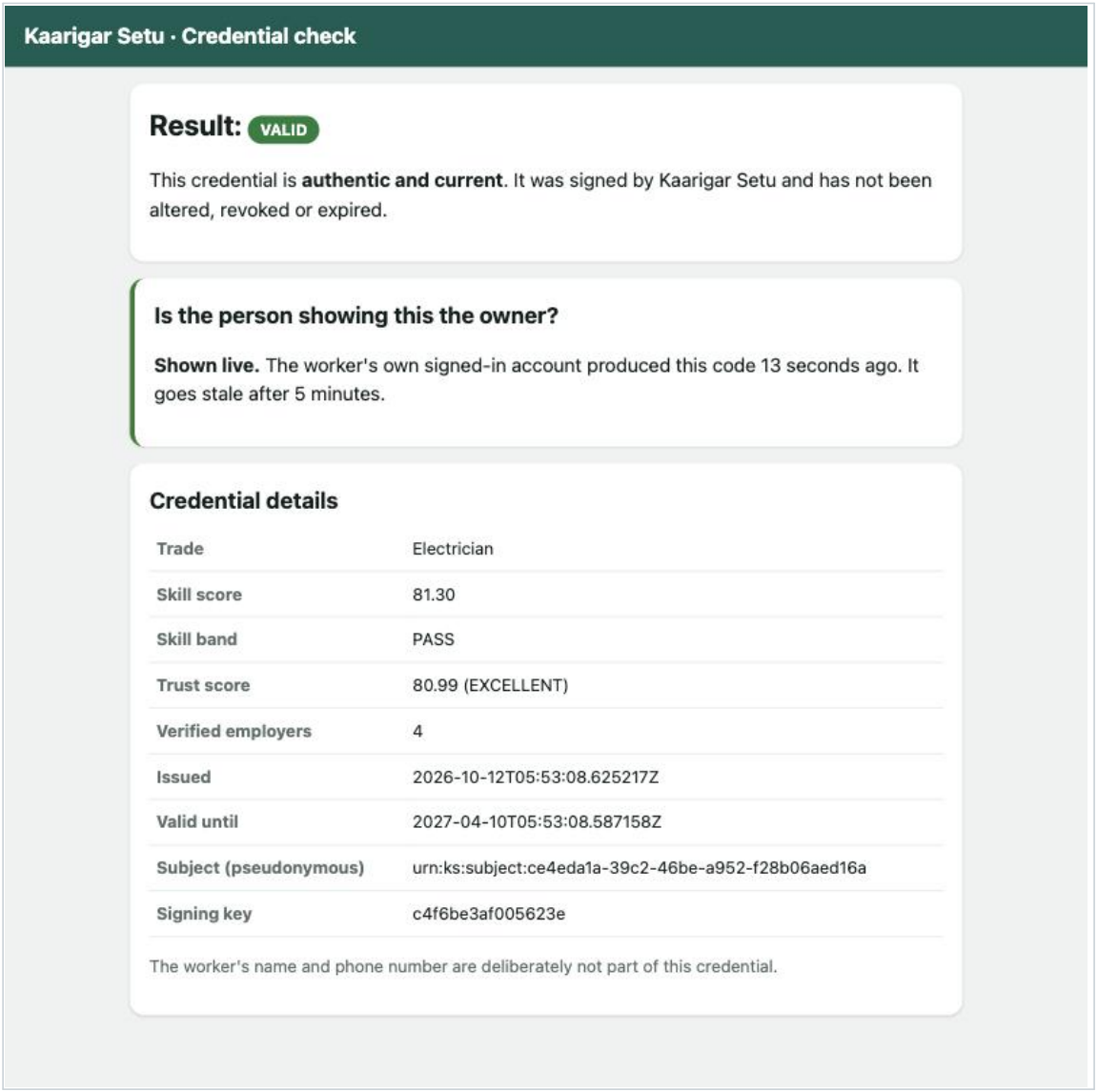
Figure 7. Public verification page: fixed link



What it shows. What a stranger sees after scanning the plain QR: result VALID, a note that a fixed link does not prove who is showing it, and the credential details, including the pseudonymous subject and the signing key id.

Implementation point. Verification is public, throttled to 120 requests a minute per address, and re-checks everything on the server. The warning box is the honest answer to screenshot replay.

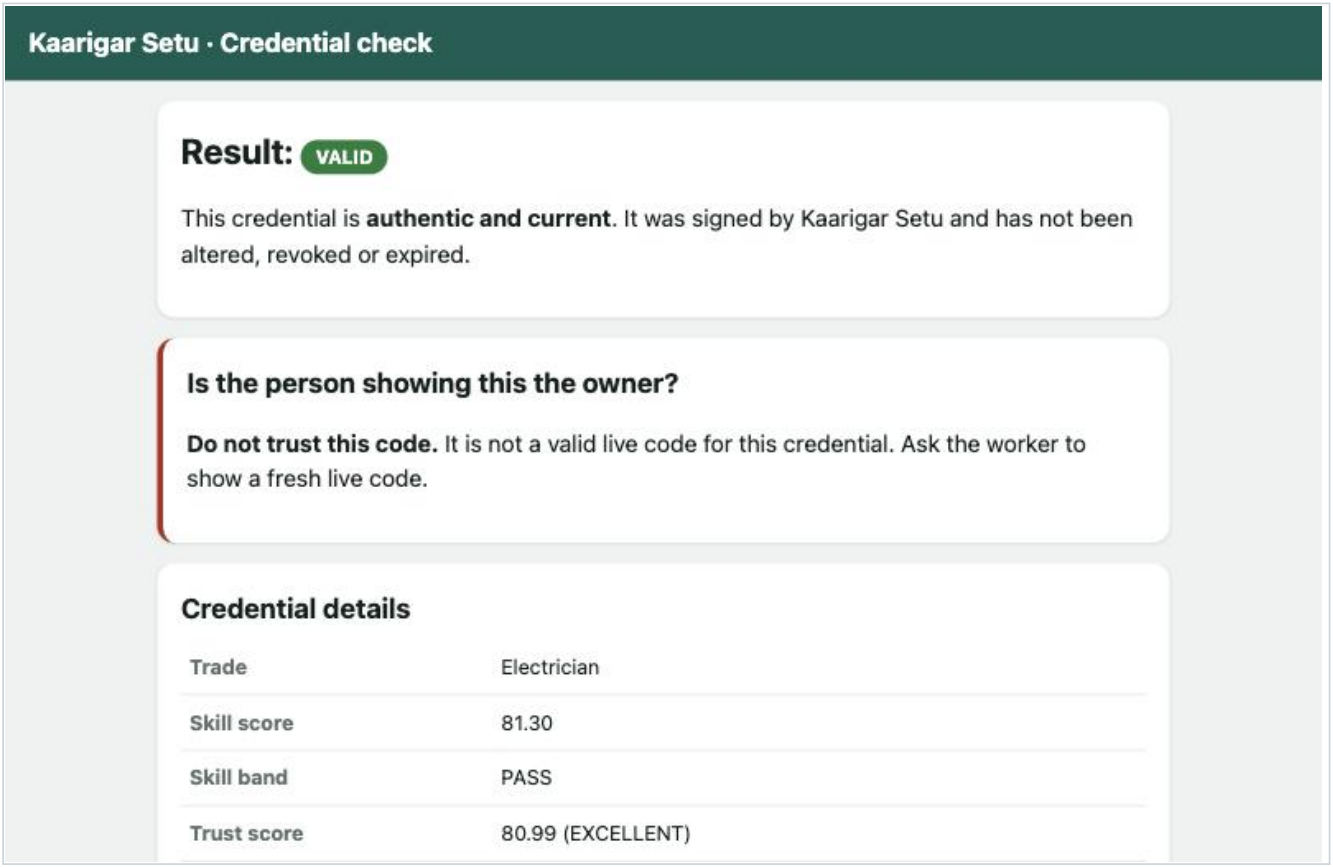
Figure 8. Public verification page: live code



What it shows. The same credential opened through a real live code: 'Shown live', produced 13 seconds ago by the worker's own signed-in account.

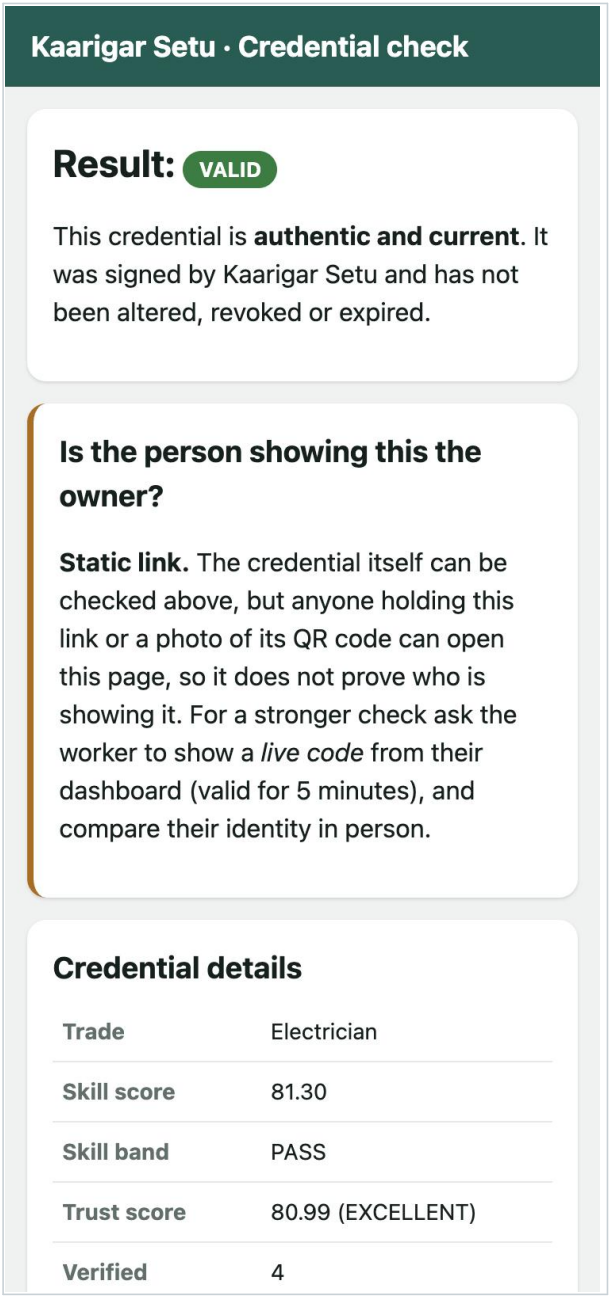
Implementation point. The link was minted through the API, decoded from the live QR and opened here; the page recomputes the HMAC and the age.

Figure 9. Public verification page: made-up code



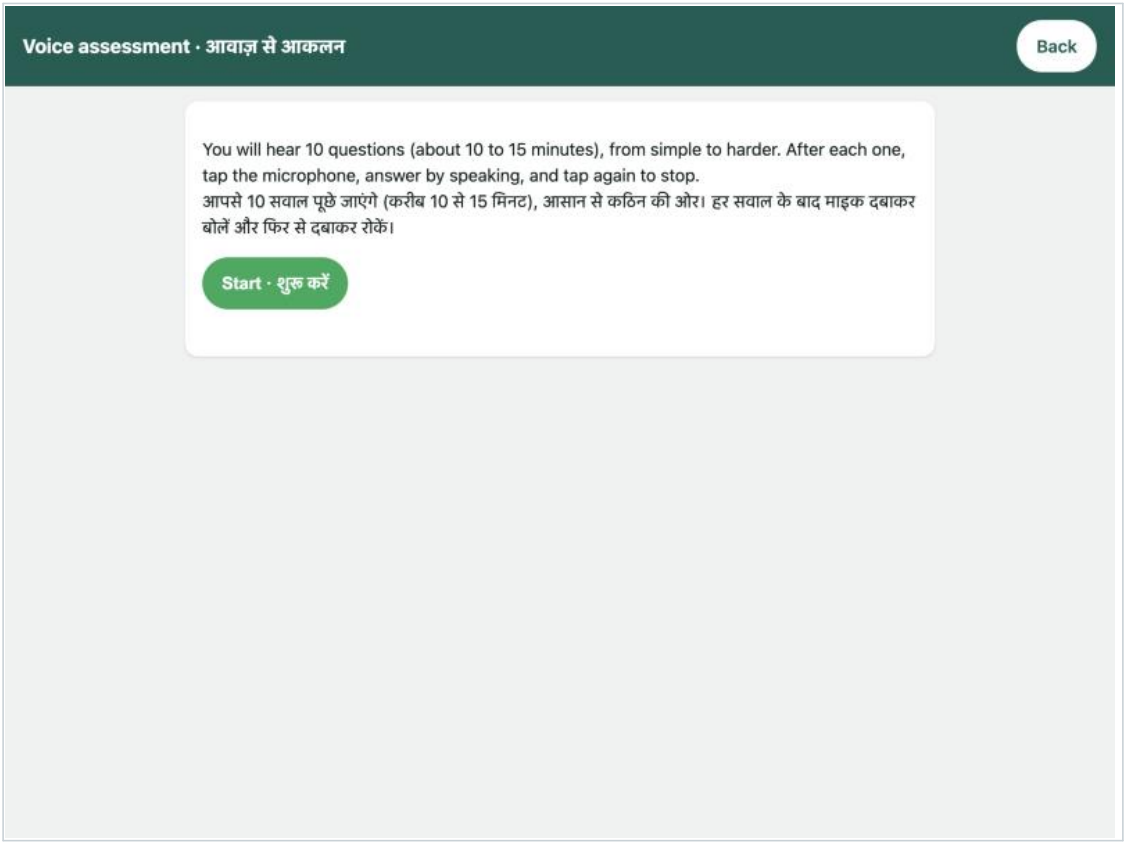
What it shows. A forged live code in the link: the credential is still VALID, but the page says 'Do not trust this code'.
Implementation point. A token that is not a valid HMAC for this credential is rejected; an old token shows 'Not live', a missing one shows 'Static link'.

Figure 10. Verification page at phone width



What it shows. The same page in a 375-pixel-wide phone layout.
Implementation point. Responsive layout with a strict content-security policy (no inline scripts) so the page works on a phone browser.

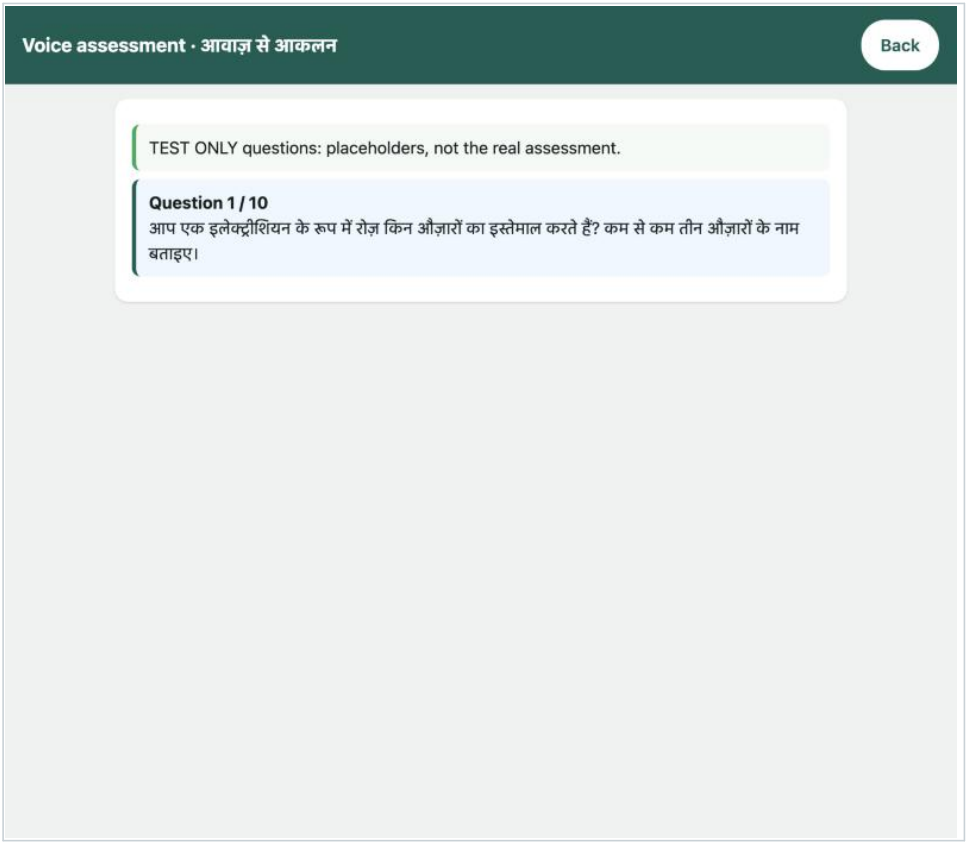
Figure 11. Voice assessment: introduction



What it shows. The bilingual introduction: ten questions, from simple to harder, tap the microphone, answer by speaking, tap again to stop.

Implementation point. While preparing these screenshots a mismatch was found and fixed: the Hindi sentence said 5 questions while the English said 10. This capture is after the fix.

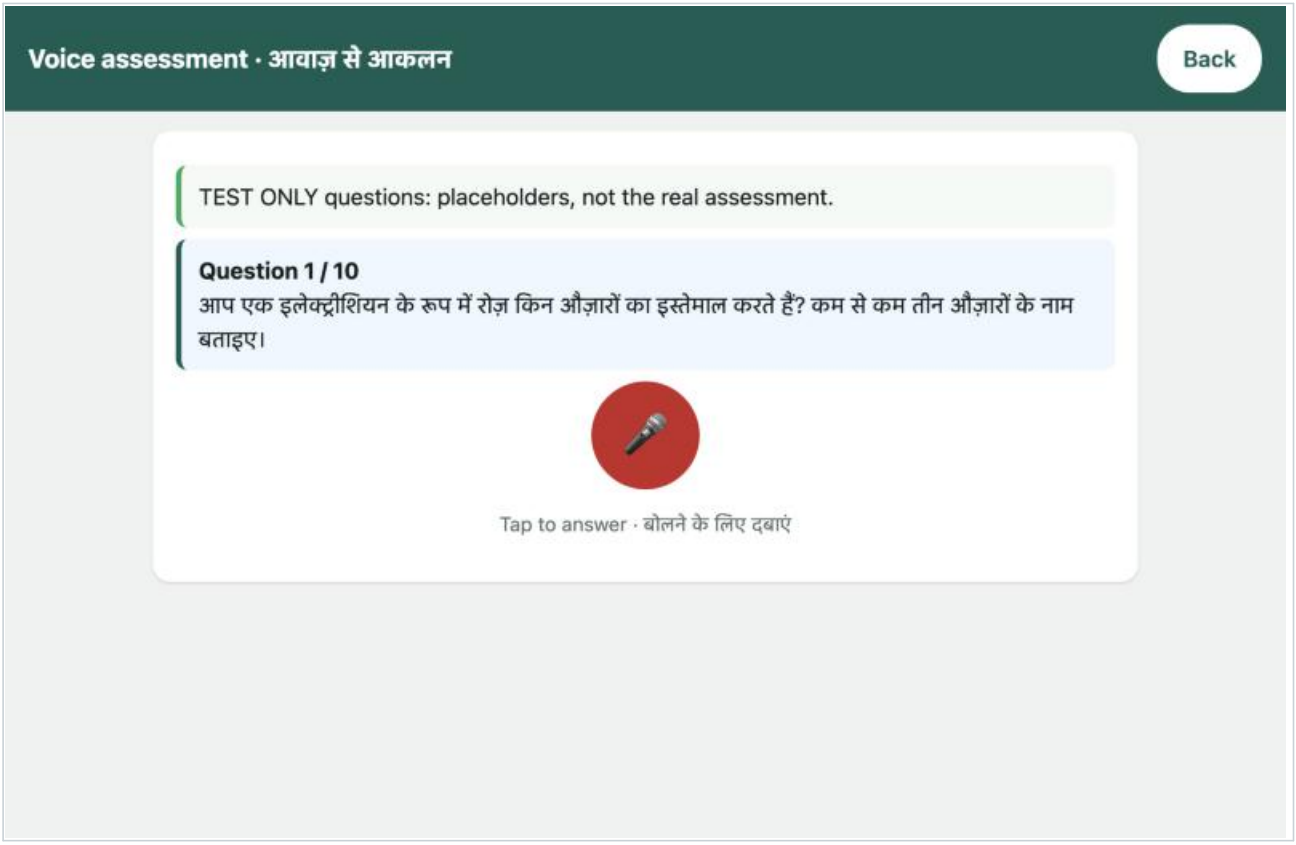
Figure 12. Voice assessment: question read aloud



What it shows. Question 1 of 10 in Hindi, with the notice that the questions are TEST ONLY placeholders.

Implementation point. Only the question text and marks are sent to the browser, never the grading keywords. The rubrics are placeholders until a trade expert supplies real ones.

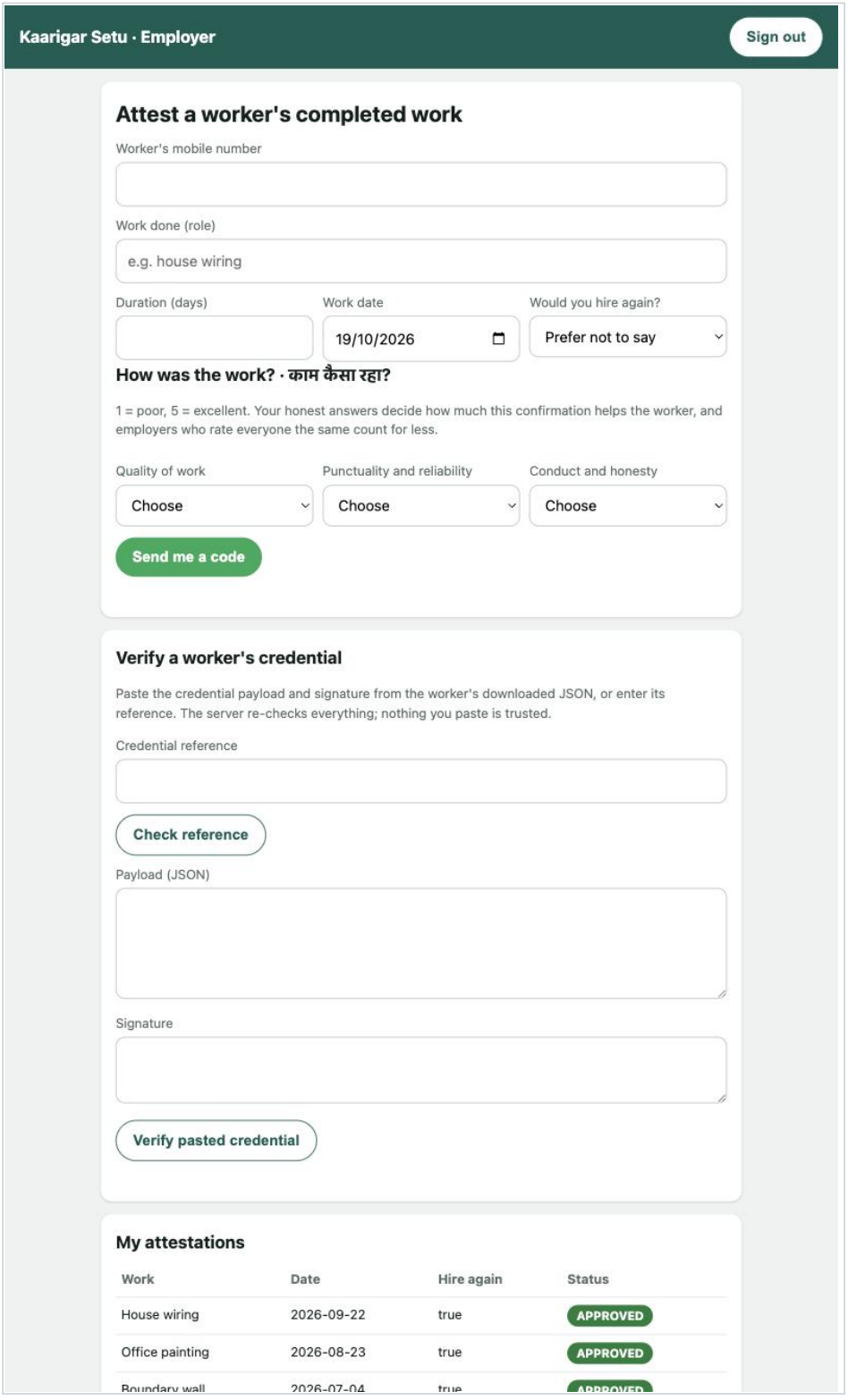
Figure 13. Voice assessment: ready to answer



What it shows. The microphone button and the prompt 'Tap to answer'.

Implementation point. The recording is converted in the browser to 16 kHz WAV, then size, header and duration are validated on the server before anything else; the audio is held in memory only and discarded after transcription.

Figure 14. Employer dashboard



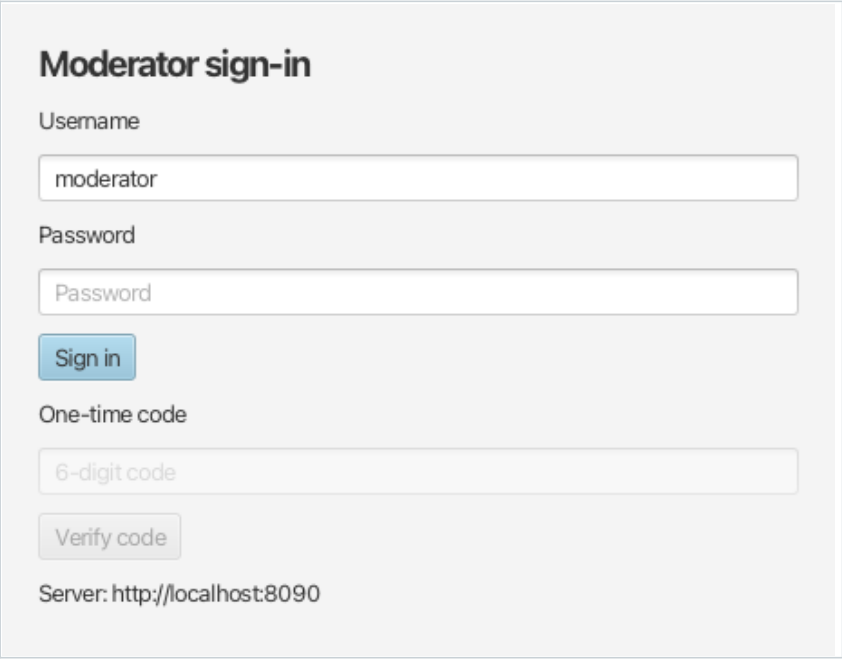
What it shows. The form to attest a worker's completed work (job, duration, date, would-hire-again, and 1 to 5 ratings for quality, punctuality and conduct), a box to verify a worker's credential, and the employer's past attestations.

Implementation point. A confirmation needs a one-time code sent to the employer's own phone, then passes the fraud model; self-attestation is refused and volume is rate-limited.

Part B. The moderator console (JavaFX desktop application)

The human-review tool for fraud flags, employer vetting and security monitoring. Rendered with the console's screenshot mode and fictional DEMO data.

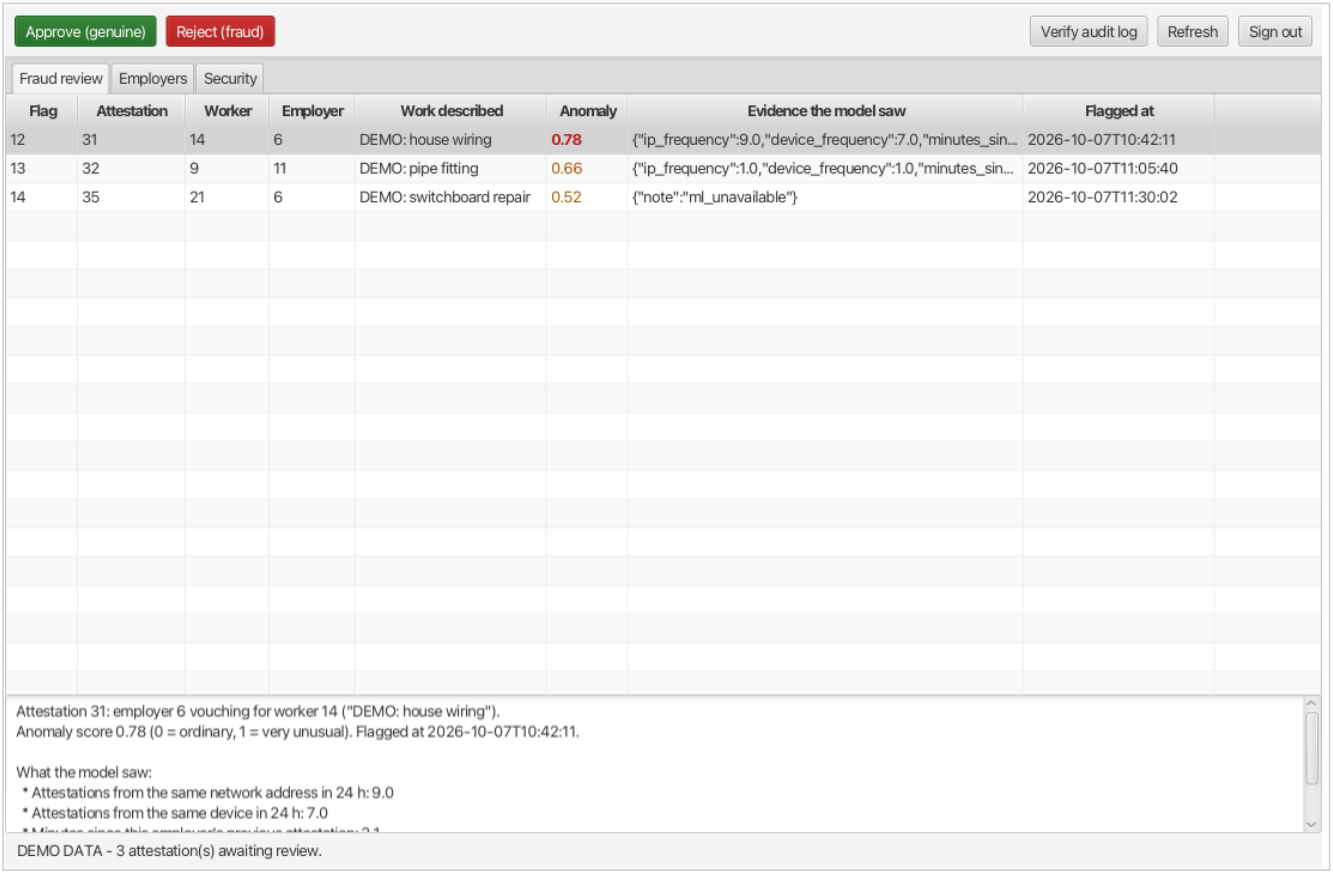
Figure 15. Console sign-in



What it shows. Two-step moderator sign-in: password, then a one-time code.

Implementation point. Password hashed with PBKDF2 (210,000 rounds); unknown usernames cost the same time as wrong passwords; five attempts per ten minutes.

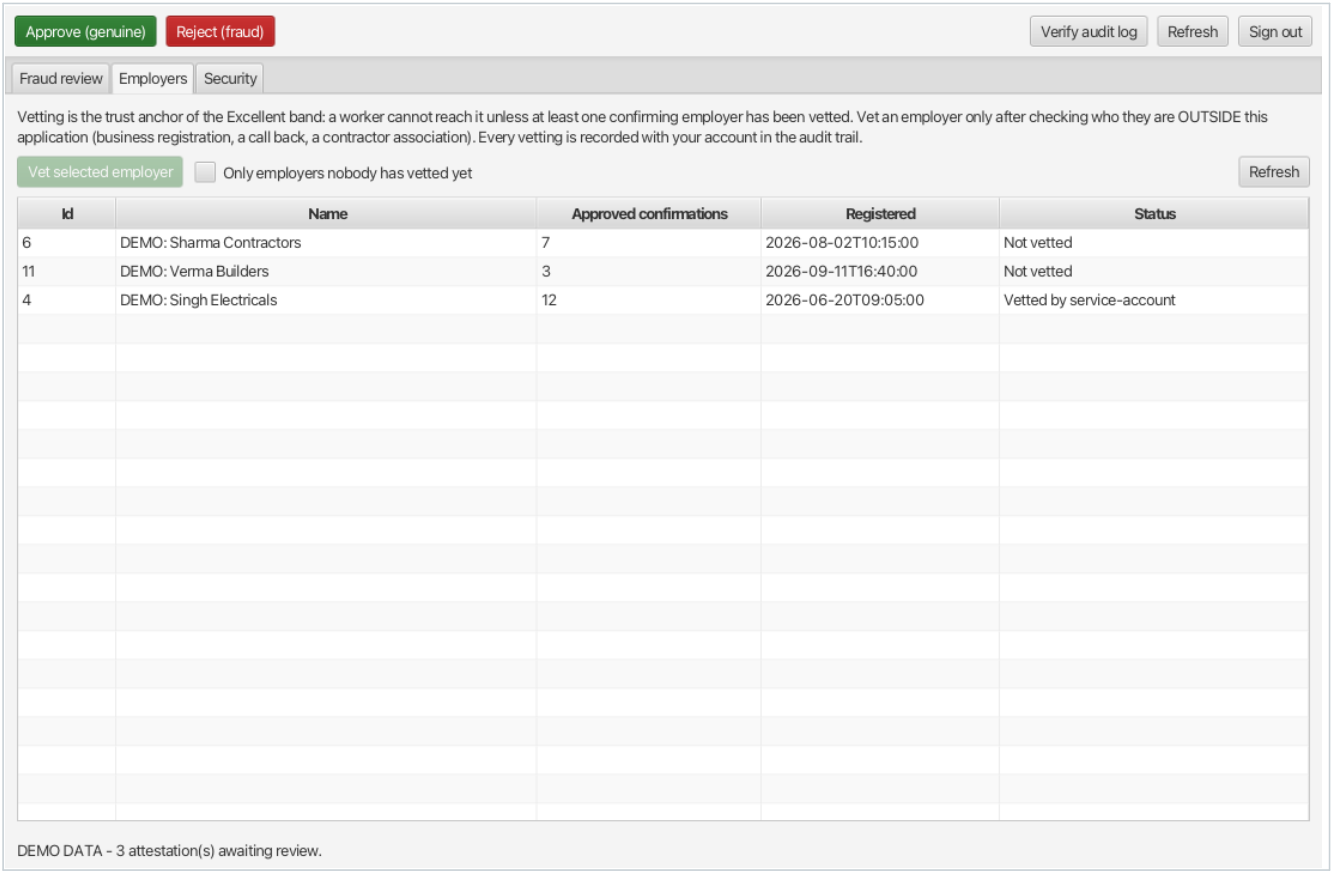
Figure 16. Fraud review queue



What it shows. Attestations the Isolation Forest flagged, with the anomaly score and the evidence the model saw; approve or reject.

Implementation point. Human in the loop for the fraud model. A flag can be decided only once and every decision is written to the audit trail.

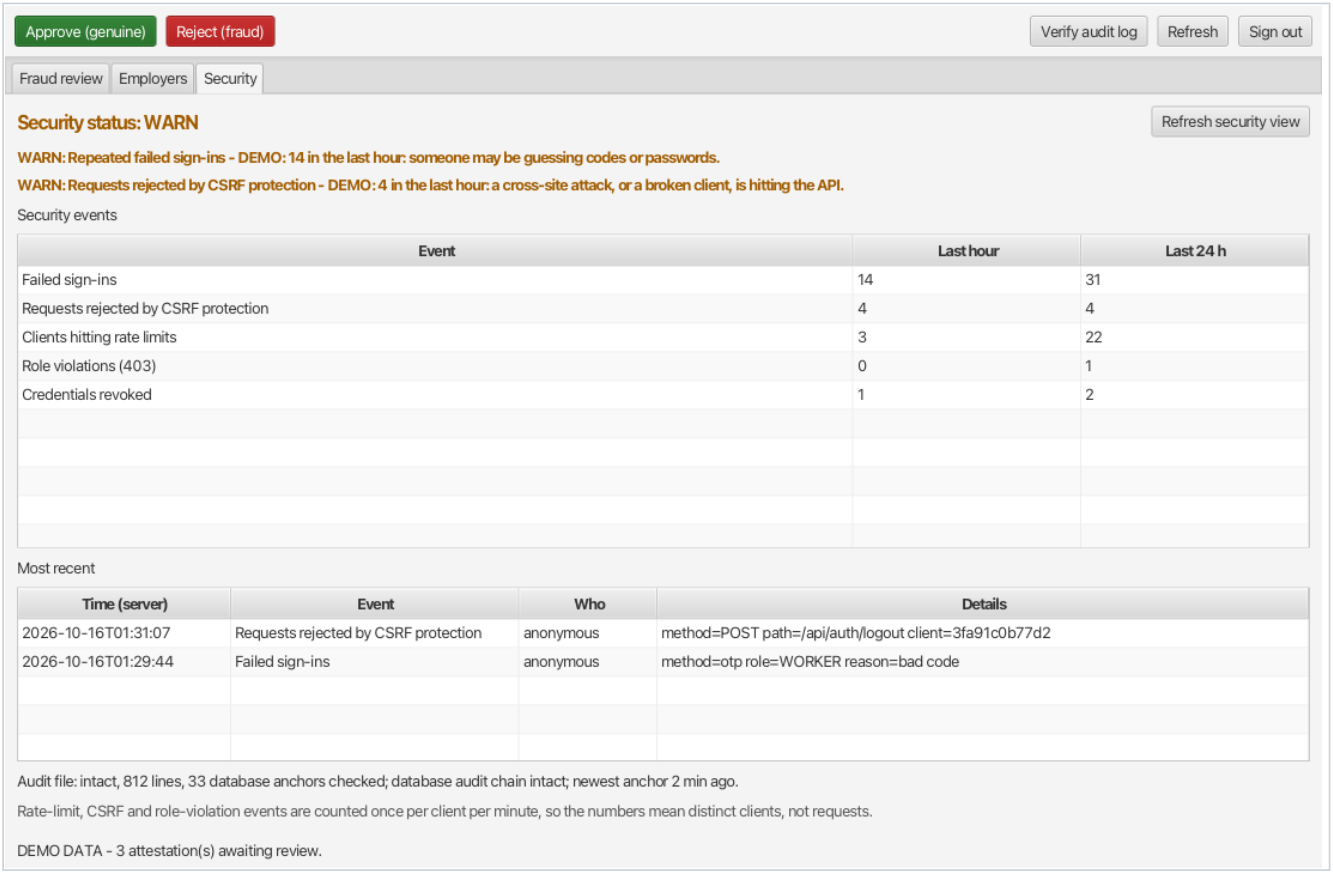
Figure 17. Employers tab: vetting



What it shows. Employers with their approved confirmations and whether they have been vetted; the explanation of what vetting means; the 'Vet selected employer' action.

Implementation point. Vetting is the trust anchor of the Excellent band; it records who and when and refreshes trust scores. This closed the weakness where three fake employer accounts could reach the top band.

Figure 18. Security tab



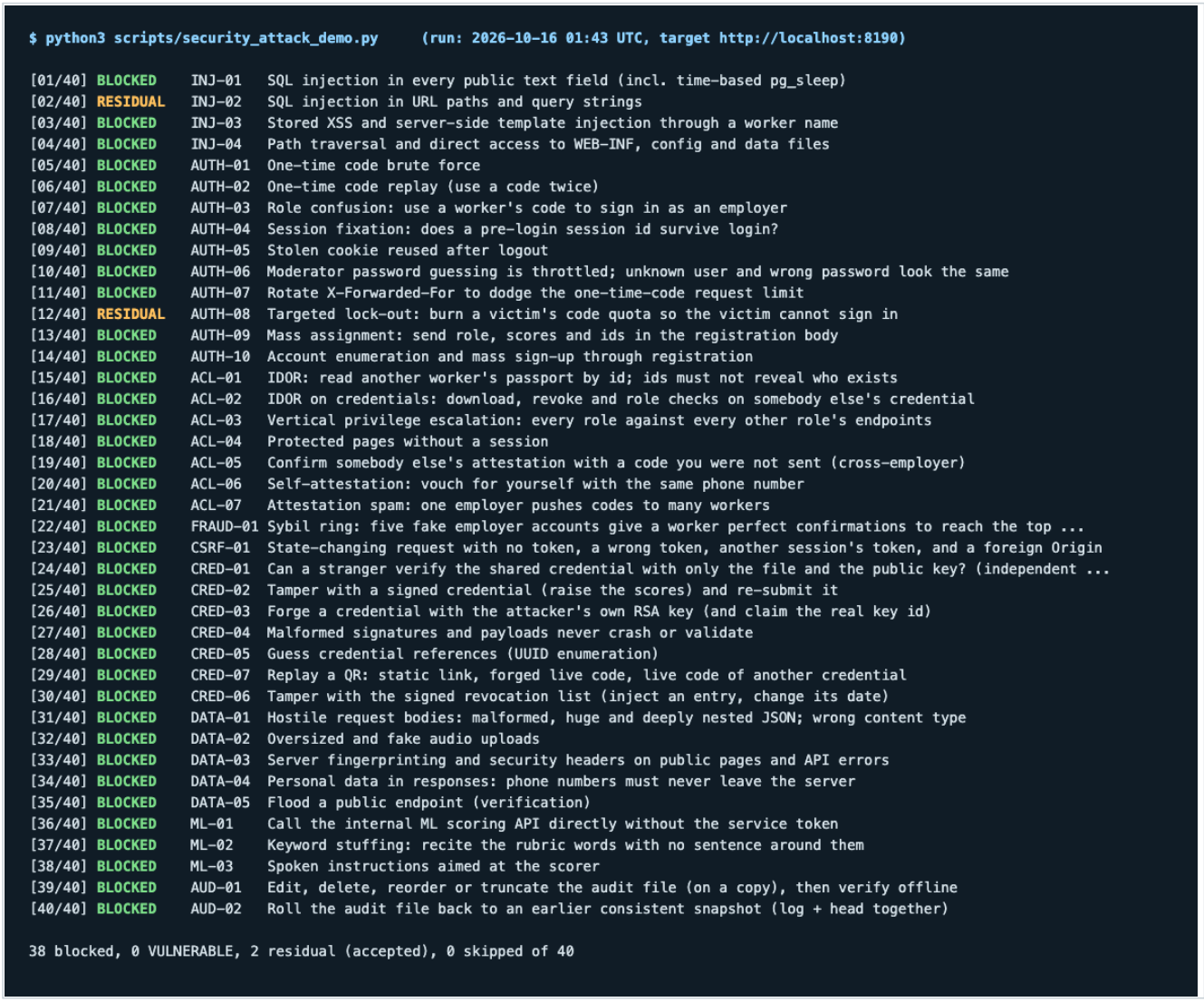
What it shows. Overall status, alerts, counts of security events for the last hour and day, the most recent events, and the audit-trail integrity line.

Implementation point. CSRF rejections, role violations and rate-limit hits are audited (once per client per minute) and compared with thresholds; the integrity line checks the audit file against the database anchors.

Part C. Command-line evidence

Real output of the project's security tooling. Dark panels are text drawn into an image.

Figure 19. Attack-and-defence run: 40 attacks



What it shows. Every attack with its outcome: 38 blocked, 2 residual (a known, documented trade-off), 0 vulnerable, 0 skipped.

Implementation point. scripts/security_attack_demo.py attacks the running stack: injection, sessions, access control, CSRF, credential forgery, data exposure, scoring abuse and audit tampering. Full details in docs/PENTEST_REPORT.md.

Figure 20. Offline credential verification: passes

```
$ python3 scripts/verify_credential.py credential.json --server http://localhost:8190
credential urn:uuid:26214185-68d1-418a-9cd2-280d360ae6f4
  [ OK ] document matches the signed text
  [ OK ] signature (RSA PKCS#1 v1.5, SHA-256)
  [ OK ] key id: public key c4f6be3af005623e, proof says c4f6be3af005623e
  [ OK ] key id inside the signed data agrees: c4f6be3af005623e
  [ OK ] validity period: 2026-10-12T05:53:08.587158Z .. 2027-04-10T05:53:08.587158Z
  [ OK ] revocation list signature
  [ OK ] revocation list is recent: 0.0 h old (up to 5 minutes of clock difference is allowed)
  [ OK ] not revoked
RESULT: all checks passed
exit status: 0
```

What it shows. An independent script checks a downloaded credential with only the file and the public key: signature, key id, validity dates and the signed revocation list.

Implementation point. scripts/verify_credential.py shares no code with the application. Signing uses RFC 8785 canonical form so any JSON tool can rebuild the signed bytes.

Figure 21. Offline credential verification: edited credential rejected

```
$ python3 scripts/verify_credential.py credential_edited.json --server http://localhost:8190    # skillScore changed to 99.9
credential urn:uuid:26214185-68d1-418a-9cd2-280d360ae6f4
  [FAIL] document matches the signed text: the shown credential differs from the text that was signed
  [ OK ] signature (RSA PKCS#1 v1.5, SHA-256)
  [ OK ] key id: public key c4f6be3af005623e, proof says c4f6be3af005623e
  [ OK ] key id inside the signed data agrees: c4f6be3af005623e
  [ OK ] validity period: 2026-10-12T05:53:08.587158Z .. 2027-04-10T05:53:08.587158Z
  [ OK ] revocation list signature
  [ OK ] revocation list is recent: 0.0 h old (up to 5 minutes of clock difference is allowed)
  [ OK ] not revoked
RESULT: FAILED
exit status: 1
```

What it shows. The same check after the score in the file was changed to 99.9: it fails and the exit status is 1.

Implementation point. Any edit to a signed field breaks verification, whether it is made to the displayed credential or to the signed text.

Figure 22. Audit trail attacks

```
$ python3 scripts/security_attack_demo.py --only AUD
Attacking http://localhost:8190  (dev stack only)

[01/2] BLOCKED    AUD-01   Edit, delete, reorder or truncate the audit file (on a copy), then verify offline
           untouched copy: INTACT lines=2333. edit one event: TAMPERED line=1168 of 2333 reason=hash chain broken; delete one line:
TAMPERED line=1167 of 2332 reason=sequence gap or reorder; swap two lines: TAMPERED line=1167 of 2333 reason=sequence gap or reorder;
cut off the last 3 lines: TAMPERED line=2330 of 2330 reason=head mismatch: lines truncated or log rewritten; edit + recompute the
whole chain (no key): TAMPERED line=2333 of 2333 reason=head mismatch: lines truncated or log rewritten
[02/2] BLOCKED    AUD-02   Roll the audit file back to an earlier consistent snapshot (log + head together)
           an older log+head pair passes the chain check alone (INTACT lines=2333) but is rejected once the anchors pinned in the
database are consulted: TAMPERED line=2333 of 2333 reason=file has 2333 lines but the database anchored line 2383: rolled back or
truncated

2 blocked, 0 VULNERABLE, 0 residual (accepted), 0 skipped of 2
```

What it shows. Edits, deletions, reordering, truncation and a consistent whole-file rewrite are detected; so is a rollback to an older matching log and head record.

Implementation point. Hash chain + keyed head record + anchors of the newest line pinned in the append-only database every 25 events and every 5 minutes.

Figure 23. Fake-employer simulation

```
$ java ... com.kaarigarsetu.trust.TrustAttackSimulation
| What the attacker did to each fake employer | Fake employers needed for Good (65) | for Excellent (80): before the vetting rule |
for Excellent (80): now |
|---|---|---|---|
| brand-new, unverified account | 3 employers | 3 employers | never (30 tried) |
| has already given 5 approved confirmations (to other fake workers) | 3 employers | 3 employers | never (30 tried) |
| warmed up AND maximum PageRank standing | 3 employers | 3 employers | never (30 tried) |
| also vetted by a moderator (needs an insider, a bribe or a breach of the vetting process) | 3 employers | 3 employers | 3 employers
|

One employer confirming many times (best case for the attacker, vetted insider):
  1 confirmations -> 41.91
  5 confirmations -> 46.86
  20 confirmations -> 49.04
  100 confirmations -> 49.80
A moderator rules one of three throwaway confirmations fraudulent:
  before: 79.99   after: 74.16
```

What it shows. How many fake employer accounts an attacker needs to reach each band, before the vetting rule and now.
Implementation point. Run against the production scoring code. Three throwaway accounts used to be enough for Excellent; now the top band is unreachable without a moderator-vetted employer.

Figure 24. Public key list and signed revocation list

```
$ curl -s http://localhost:8190/api/credential/keys   (public key shortened)
{
  "keys": [
    {
      "keyId": "c4f6be3af005623e",
      "status": "active",
      "algorithm": "RSA-3072",
      "publicKeyPem": "-----BEGIN PUBLIC KEY----- MIIBojANBgkqhkiG9w0BAQEFAAOCAY8AM ..."
    }
  ]
}

$ curl -s http://localhost:8190/api/credential/revocations
{
  "list": {
    "type": "KaarigarSetuRevocationList",
    "issuedAt": "2026-10-19T17:44:21.622707Z",
    "keyId": "c4f6be3af005623e",
    "revoked": [
      {
        "ref": "4fd460ae-3e21-4dd0-b5ea-cf99cccd8dba",
        "revokedAt": "2026-10-16T01:43:42.559033Z"
      }
    ]
  },
  "proof": {
    "type": "RsaSignature2018-PKCS1v15-SHA256",
    "keyId": "c4f6be3af005623e",
    "canonicalization": "RFC8785-JCS",
    "signature": "qlgwK+18VeWlgVhBRc6EagSJ3ijd28Bbva3zsiDIw7jk..."
  }
}
```

What it shows. The endpoints a verifier uses: the active signing key and the signed list of revoked credentials.
Implementation point. Retired keys stay listed after rotation so old credentials keep verifying; the revocation list is signed so a proxy cannot add, remove or re-date entries.

Figure 25. Speech recognition (8-bit Whisper) and the service token

```
$ curl -X POST "http://127.0.0.1:8101/internal/ml/transcribe?language=hi" -H "X-Internal-Token: <secret>" -F audio=@clip.wav
  (a spoken Hindi sentence, recognised by the 8-bit Whisper model)
{"transcript":"मैं रोज पेच का स्प्लायर और टेस्टर का इस्तेमाल करता हूँ।","language":"hi"}

$ same call without the token
{"detail":"invalid internal token"}
```

What it shows. A spoken Hindi sentence recognised by the memory-light Whisper engine, and the same call without the secret token being refused.

Implementation point. The internal scoring service only answers callers that present the shared token. Measured accuracy on real voices is lower than on this clean synthetic voice (about a third of words wrong); see docs/ACCENT_TRAINING.md.

Figure 26. Test suites

```
$ cd kaarigarsetu-java && mvn test
Tests run: 5, Failures: 0, Errors: 0, Skipped: 0  in credential.QRGeneratorTest
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0  in credential.SignatureKeyRotationTest
Tests run: 13, Failures: 0, Errors: 0, Skipped: 0  in credential.CredentialServiceTest
Tests run: 5, Failures: 0, Errors: 0, Skipped: 0  in util.WavValidatorTest
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0  in util.CanonicalJsonTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0  in util.FileUtilitiesTest
Tests run: 7, Failures: 0, Errors: 0, Skipped: 0  in security.PresentationTokensTest
Tests run: 9, Failures: 0, Errors: 0, Skipped: 0  in security.ChainedAuditFileSinkTest
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0  in security.PasswordHasherTest
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0  in security.RateLimiterTest
Tests run: 7, Failures: 0, Errors: 0, Skipped: 0  in security.SignatureServiceKeystoreTest
Tests run: 8, Failures: 0, Errors: 0, Skipped: 0  in security.OtpServiceTest
Tests run: 12, Failures: 0, Errors: 0, Skipped: 0  in security.AuthServiceTest
Tests run: 7, Failures: 0, Errors: 0, Skipped: 0  in security.PhoneProtectorTest
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0  in security.OtpAndSignatureServiceTest
Tests run: 10, Failures: 0, Errors: 0, Skipped: 0  in security.AuditLoggerTest
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0  in security.HttpSmsOtpSenderTest
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0  in web.SessionsTest
Tests run: 9, Failures: 0, Errors: 0, Skipped: 0  in web.AuthFilterTest
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0  in web.CsrfFilterTest
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0  in web.StaticAssetsTest
Tests run: 5, Failures: 0, Errors: 0, Skipped: 0  in moderator.SecurityMonitorTest
Tests run: 9, Failures: 0, Errors: 0, Skipped: 0  in moderator.ModeratorServiceTest
Tests run: 13, Failures: 0, Errors: 0, Skipped: 0  in trust.TrustScoreServiceTest
Tests run: 31, Failures: 0, Errors: 0, Skipped: 0  in trust.ReputationScoringTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0  in trust.TrustGraphServiceTest
Tests run: 5, Failures: 0, Errors: 0, Skipped: 0  in trust.TrustAttackSimulationTest
Tests run: 322, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS

$ cd kaarigarsetu-moderator-console && mvn test
Tests run: 13, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS

$ cd kaarigarsetu-ml-service && python -m pytest -q
370 passed in 2.77s
```

What it shows. Java (322), console (13) and Python (370) test results, with the security-related test classes listed.

Implementation point. Tests include the guard that fails the build if any database query is built by joining strings, the SQL-injection fuzz over every data-access method, key rotation, audit anchors and the Sybil simulation.

Figure 27. Static analysis, secret and dependency scans

```
$ mvn -Psast compile spotbugs:check      # SpotBugs + find-sec-bugs
exit code: 0 (0 = no findings; 73 pattern-noise items excluded with reasons in spotbugs-exclude.xml)

$ bandit -q -r kaarigarsetu-ml-service/app -ll
exit code: 0

$ semgrep scan --config p/java --config p/python --config p/javascript --config p/owasp-top-ten --config p/secrets --error ...
exit code: 0 (0 = no findings in 213 files)

$ gitleaks dir <tree that ships in the zip> -c .gitleaks.toml
11:15PM INF no leaks found

$ pip-audit -r requirements.txt  /  -r requirements-faster-whisper.txt
No known vulnerabilities found
No known vulnerabilities found

$ python3 scripts/osv_scan_maven.py
checked 15 dependencies, 0 problem(s)
```

What it shows. SpotBugs with security rules, Bandit, Semgrep, a secret scan of what ships and the dependency scans, all clean.

Implementation point. Findings from earlier runs were fixed or justified in writing (docs/SECURITY_SCAN_REPORT.md). The same checks are defined as CI jobs, which have not yet run on GitHub.

Figure 28. End-to-end suite: the security sections

```
$ STT_PROVIDER=stub ./scripts/dev.sh  &&  python3 scripts/e2e.py
(excerpt: the new security sections)
== 7. Credential: public verification, QR, tamper, revoke
  [PASS] public verify (no login) -> VALID
  [PASS] the signed credential carries the skill band
  [PASS] credential never contains name or phone
  [PASS] QR code is a PNG
  [PASS] verify page renders 'VALID'
  [PASS] downloaded signed document has a proof
  [PASS] public key list names the signing key as active
  [PASS] the credential is signed in canonical form (RFC 8785)
  [PASS] independent offline verifier accepts it (file + public key only)
  [PASS] independent offline verifier rejects an edited score
  [PASS] a re-written copy of the file (new whitespace and order) still verifies offline
  [PASS] TAMPERED payload fails verification
  [PASS] garbage signature fails verification (no exception)
  [PASS] anonymous cannot mint a live code -> 401
  [PASS] employer cannot mint a live code -> 403
  [PASS] another worker cannot mint a live code for my credential -> 404
  [PASS] owner gets a live-code QR image
  [PASS] without a code the verification says presentation NONE
  [PASS] the verify page for the fixed link says it is a static link
  [PASS] a made-up live code is flagged as not trustworthy
  [PASS] employer cannot revoke -> 403
  [PASS] owner can revoke
  [PASS] revoked credential -> REVOKED
  [PASS] the signed revocation list names it
  [PASS] offline verifier rejects it once the revocation list is consulted
  [PASS] a fresh credential can be issued
  [PASS] new credential VALID
  [PASS] another worker cannot download my credential -> 404

== 9. Moderator login (password + OTP) and decision
  [PASS] wrong moderator password -> 401
  [PASS] moderator password accepted, OTP step required
  [PASS] password alone does NOT grant moderator access
  [PASS] moderator OTP -> logged in
  [PASS] flagged queue is not empty
  [PASS] queue exposes no phone numbers or names
  [PASS] moderator approves a flagged attestation
  [PASS] a flag cannot be decided twice -> 409
  [PASS] moderator rejects another as fraud
  [PASS] invalid decision value -> 400
  [PASS] audit file hash chain verifies
  [PASS] audit check also compares the file with the anchors kept in the database
  [PASS] security monitor answers for a moderator
  [PASS] it saw the CSRF rejections from earlier in this run
  [PASS] it saw the role violations from earlier in this run
  [PASS] it lists recent security events without phone numbers
  [PASS] a worker cannot read the security monitor -> 403

== 9b. A ring of fake employers: five unvetted accounts cannot give the top band, one moderator vetting can
  [PASS] the five fake employers' confirmations all pass OTP and the fraud model
  [PASS] five unvetted employers with perfect ratings: trust is held at exactly 79.99 (the hold is what stops it, not the ratings)
  [PASS] the worker is told why
  [PASS] the moderator's vetting list shows the unvetted employers without phone numbers
  [PASS] a worker cannot vet an employer -> 403
  [PASS] a moderator can vet an employer
  [PASS] one vetted employer among the five lifts the hold: Excellent
  [PASS] vetting a non-existent employer -> 404


PASS lines: 108   FAIL lines: 0
ALL CHECKS PASSED
```

What it shows. Excerpt of the 108-check end-to-end run covering credential verification, the moderator flow and the fake-employer ring that is held below Excellent until one employer is vetted.

Implementation point. Runs against the live application with the labelled test transcriber, so the voice step uses synthetic audio rather than real speech.